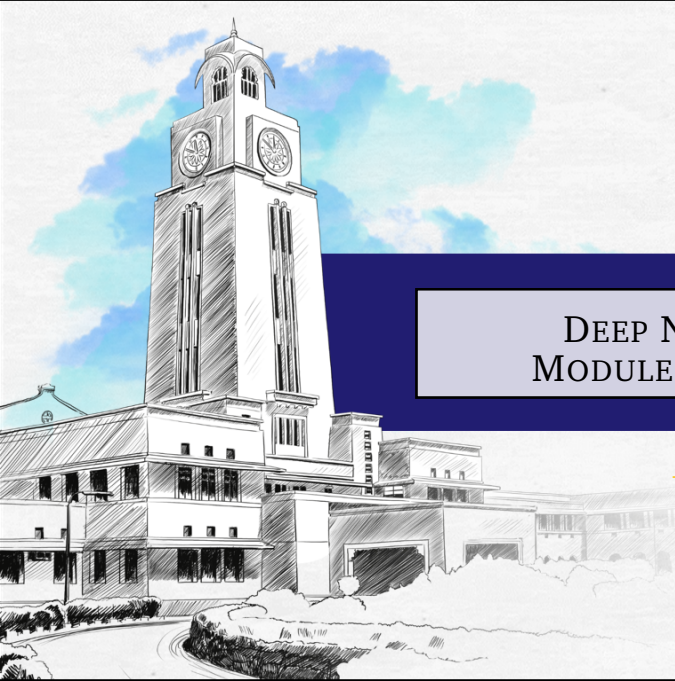


BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Deep Neural Network
Module # 10: Optimizers

Seetha Parameswaran
BITS Pilani WILP

The instructor is gratefully acknowledging
the authors who made their course
materials freely available online.
This deck is prepared by Seetha Parameswaran.

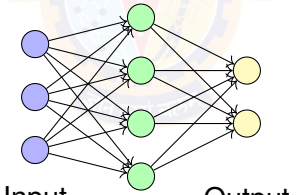
Table of Contents

Why Optimization Matters

- Machine Learning Goal: Find parameters θ^* that minimize loss

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

- Deep neural networks have millions to billions of parameters
- No closed-form solution $\Rightarrow$ need iterative optimization algorithms
- Without optimization: random weights $\Rightarrow$ random predictions $\Rightarrow$ no learning



$$\theta = \{W_1, b_1, W_2, b_2, \ldots\}$$

Gradient Descent: One Iteration Example

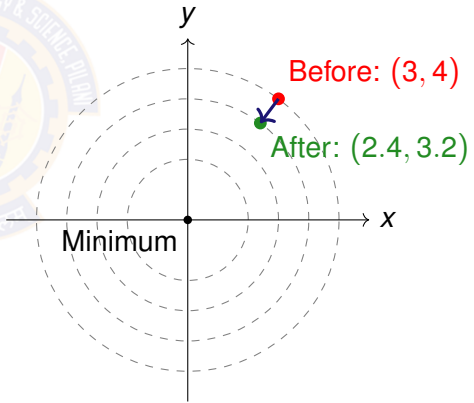
- Simple 2D function: $f(x, y) = x^2 + y^2$
- Gradient: $\nabla f = [2x, 2y]$
- Starting point: $(x_0, y_0) = (3, 4)$, Learning rate $\eta = 0.1$

Before Update:

- Position: $(3, 4)$
- Loss: $f(3, 4) = 9 + 16 = 25$
- Gradient: $\nabla f = [6, 8]$

After Update:

- $x_1 = 3 - 0.1(6) = 2.4$
- $y_1 = 4 - 0.1(8) = 3.2$
- Loss: $f(2.4, 3.2) = 16$ (36% reduction!)



Loss Functions Review

Loss Function	Formula	Gradient	Use Case
Mean Squared Error	$\frac{1}{2}(\hat{y} - y)^2$	$(\hat{y} - y)$	Regression tasks
Mean Absolute Error	$\lvert\hat{y} - y\rvert$	$\text{sign}(\hat{y} - y)$	Regression with outliers
Binary Cross-Entropy	$-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	$(\hat{y} - y)$	Binary classification
Categorical Cross-Entropy	$-\sum_k y_k \log \hat{y}_k$	$(\hat{\mathbf{y}} - \mathbf{y})$	Multi-class classification

- Key observation: All gradients have form (prediction − target) or similar
- Simple gradients enable efficient backpropagation
- Choice of loss function depends on problem type and data characteristics

Optimization Challenges - Concepts

- Saddle Points:
  - Gradient $\nabla \mathcal{L} = 0$ but not a minimum
  - Point where some directions go up, others go down
  - Common in high-dimensional spaces
- Plateaus:
  - Flat regions with very small gradients
  - Learning becomes very slow
  - Hard to determine if stuck or just slow progress
- Non-Convex Landscapes:
  - Multiple local minima
  - No guarantee of finding global minimum
  - Different initializations lead to different solutions

Optimization Challenges - Visualization

How to Identify:

- Loss curve flattens but validation error still high
- Gradient norms approach zero: $\|\nabla \mathcal{L}\| \to 0$
- Training progress stagnates for many epochs

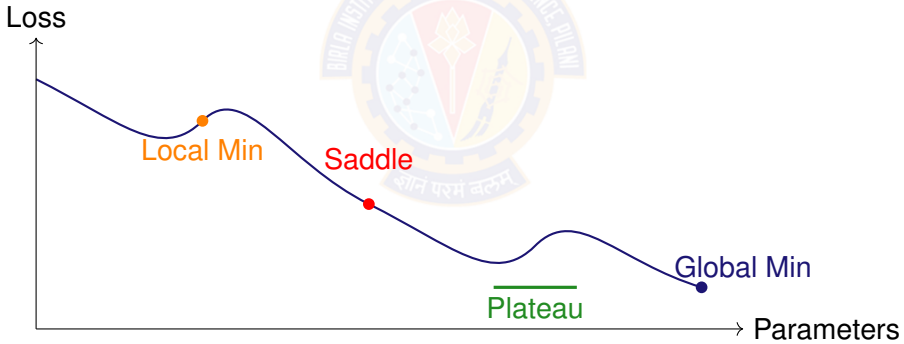
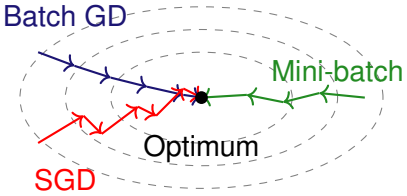


Table of Contents

Gradient Descent Variants: Comparison

Method	Gradient	Epoch	Memory	Convergence
Batch GD	$\frac{1}{N}\sum_{i=1}^{N}\nabla\mathcal{L}_i$	1	High	Smooth, slow
SGD	$\nabla\mathcal{L}_i$	N	Low	Noisy, fast
Mini-batch	$\frac{1}{B}\sum_{i=1}^{B}\nabla\mathcal{L}_i$	N/B	Medium	Balanced



- Example: Dataset with $N = 10,000$ samples, batch size $B = 32$
  - Batch GD: 1 update per epoch (all data at once)
  - SGD: $10,000$ updates per epoch (one sample at a time)
  - Mini-batch: $10,000/32 \approx 313$ updates per epoch
- Practical choice: Mini-batch with $B \in \{32, 64, 128, 256\}$
  - Balances computation speed (GPU efficiency) with convergence stability
  - More updates than Batch GD, less noise than SGD

Algorithm: Mini-Batch Gradient Descent

Algorithm 1: Mini-Batch Gradient Descent

Data: Learning rate η, batch size B, initial parameters θ

while *stopping criterion not met* **do**
  Shuffle training data
  for *each mini-batch* $\mathcal{B}$ *of size* B **do**
    Compute loss on mini-batch: $\mathcal{L}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathcal{L}_i(\theta)$
    Compute gradient: $\mathbf{g} = \nabla_\theta \mathcal{L}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \mathcal{L}_i(\theta)$
    Update parameters: $\theta \leftarrow \theta - \eta \mathbf{g}$

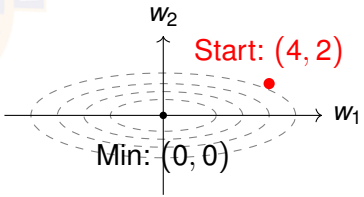
- We'll use this as our baseline for numerical examples

Toy Problem for Numerical Examples

- Loss function: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$ (elongated bowl)
- Gradient: $\nabla\mathcal{L} = [2w_1, 8w_2]$
- Initial point: $(w_1, w_2) = (4, 2)$, Loss $= 16 + 16 = 32$
- Learning rate: $\eta = 0.1$

Key Characteristics:

- w_2 direction 4× steeper than w_1
- Fixed LR causes oscillation in w_2
- Adaptive methods should help



We'll track this problem through Mini-batch GD, Momentum, and Adam

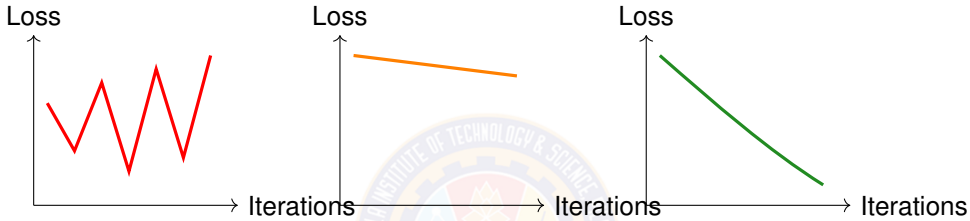
Mini-Batch GD: Numerical Example

- Problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.1$

Iter	Loss	∇_{w_1}	∇_{w_2}	$w_1 \leftarrow w_1 - \eta\nabla_{w_1}$	$w_2 \leftarrow w_2 - \eta\nabla_{w_2}$
0	32.00	8.0	16.0	$4 - 0.1(8) = 3.20$	$2 - 0.1(16) = 0.40$
1	10.88	6.4	3.2	$3.2 - 0.1(6.4) = 2.56$	$0.4 - 0.1(3.2) = 0.08$
2	6.58	5.1	0.64	$2.56 - 0.1(5.1) = 2.05$	$0.08 - 0.1(0.64) = 0.016$
3	4.21	4.1	0.13	$2.05 - 0.1(4.1) = 1.64$	$0.016 - 0.1(0.13) = 0.003$

- Observation: Loss decreases: $32 \rightarrow 10.88 \rightarrow 6.58 \rightarrow 4.21$
- Issue: w_2 takes large steps ($\Delta w_2 = -1.6$) compared to w_1 ($\Delta w_1 = -0.8$) due to steeper gradient (coefficient 8 vs 2). This creates unbalanced progress: w_2 moves too aggressively.
- This motivates: Momentum (smooth updates) and adaptive methods (per-parameter rates)

Learning Rate Effects

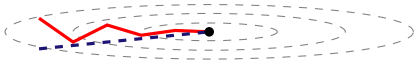


- Too large η: Oscillation, potential divergence, loss jumps around
  - Eg: $\eta = 1.0$: Loss oscillates wildly: $15.3 \rightarrow 8.2 \rightarrow 19.7$ - diverging!
- Too small η: Very slow progress, may not converge in reasonable time
  - Eg: $\eta = 0.0001$: Barely improves: $22.5 \rightarrow 21.8 \rightarrow 20.1$ - would need 1000s of iterations
- Optimal η: Fast, smooth convergence to minimum
  - Eg: $\eta = 0.01$: Smooth descent: $22.5 \rightarrow 15.2 \rightarrow 3.1$ - reaches low loss quickly

Motivation for Adaptive Techniques

- Problem with Fixed Learning Rate:
  - All parameters updated with same learning rate
  - Different parameters may need different rates
  - Some directions in loss landscape are steeper than others
- Sparse vs Dense Features:
  - Sparse features: appear rarely, need larger updates when they do
  - Dense features: appear frequently, need smaller, careful updates
- Elongated Contours:
  - Fixed LR causes slow zigzag progress
  - Need different step sizes for different directions

Fixed LR: Slow



Adaptive: Efficient

Table of Contents

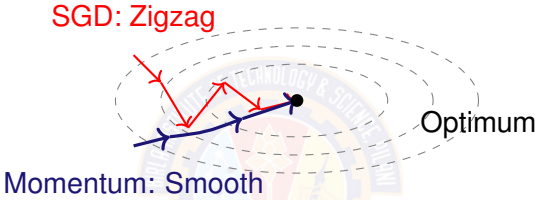
Momentum: Physical Intuition

- Analogy: Rolling ball gathering velocity down a hill
  - Ball accumulates momentum in consistent directions
  - Dampens oscillations in perpendicular directions
  - Can roll through small bumps (escape shallow local minima)
- Mathematical Formulation:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_\theta \mathcal{L}(\theta_t)$$
$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_t$$

- Key Components:
  - $\mathbf{v}_t$: velocity (accumulated gradient)
  - β: momentum coefficient, typically 0.9
  - Gradient adds to velocity, not directly to parameters

Momentum: Visualization



Benefits:

- Faster convergence than vanilla SGD
- Smooths out oscillations in ravines
- Better escape from local minima

Algorithm: SGD with Momentum

Algorithm 2: SGD with Momentum

Data: Learning rate η, momentum $\beta = 0.9$, initial parameters θ

Initialize velocity $\mathbf{v} = 0$

while *stopping criterion not met* **do**

  for *each mini-batch* $\mathcal{B}$ **do**

    Compute gradient: $\mathbf{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta)$

    Update velocity: $\mathbf{v} \leftarrow \beta \mathbf{v} + \mathbf{g}$

    Update parameters: $\theta \leftarrow \theta - \eta \mathbf{v}$

- Hyperparameter: $\beta = 0.9$ means 90% of previous velocity retained
- Additional memory: store velocity vector same size as parameters
- When to use: General deep learning, production systems

Momentum: Numerical Example

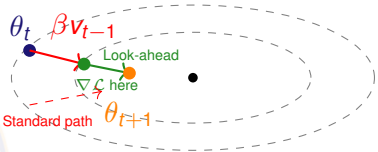
- Same problem, reduced LR: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\beta = 0.9$

Iter	Loss	$[\nabla w_1, \nabla w_2]$	$v_1 \leftarrow \beta v_1 + w_1$	$v_2 \leftarrow \beta v_2 + w_2$	w_1	w_2
0	32.00	$[8, 16]$	8	16	3.6	1.2
1	18.72	$[7.2, 9.6]$	$7.2 + 7.2 = 14.4$	$14.4 + 9.6 = 24$	2.88	0
2	8.29	$[5.76, 0]$	$13.0 + 5.76 = 18.7$	$21.6 + 0 = 21.6$	1.94	-1.08
3	8.43	$[3.88, -8.64]$	$16.9 + 3.88 = 20.7$	$19.4 - 8.64 = 10.8$	0.91	-1.62

- Still oscillates! $w_2 : 2 \rightarrow 1.2 \rightarrow 0 \rightarrow -1.08 \rightarrow -1.62$
- Key lesson: Momentum amplifies steps - needs smaller η than vanilla GD
- For remaining examples: We'll use $\eta = 0.05$ for fair comparison

Nesterov Accelerated Gradient

- Standard Momentum:
  1. Compute gradient at current θ_t
  2. Add to velocity
  3. Take step
- Nesterov Momentum:
  1. "Look ahead": jump to $\theta_t - \beta v_{t-1}$
  2. Compute gradient there
  3. Add to velocity, take step
- Benefit: More responsive to gradient changes, slightly faster convergence



Update:

$$v_t = \beta v_{t-1} + \nabla \mathcal{L}(\theta_t - \beta v_{t-1})$$
$$\theta_{t+1} = \theta_t - \eta v_t$$

From Momentum to Adaptive Methods

- Momentum solved one problem:
  - Smooths oscillations by accumulating velocity
  - Faster convergence in consistent directions
- But still has a limitation:
  - All parameters still share the same learning rate η
  - In our toy example: w_1 has small gradients, w_2 has large gradients
  - Ideally: w_1 needs larger rate, w_2 needs smaller rate
- Next: Adaptive methods
  - AdaGrad: Accumulates squared gradients $\rightarrow$ per-parameter rates
  - RMSprop: Fixes AdaGrad's shrinking learning rate
  - Adam: Combines RMSprop + Momentum = best of both worlds

Why Different Parameters Need Different Rates

- Example from our toy problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$

Parameter	Actual Gradients	Problem	Needs
w_1	$[8.0, 6.4, 5.1, 4.1]$	Moderate, decreasing	Moderate LR
w_2	$[16.0, 3.2, 0.64, 0.13]$	Large initially, drops fast	Adaptive LR

- Observation from Mini-batch GD iterations:
  - ∇_{w_2} starts 2× larger than ∇_{w_1} (16 vs 8)
  - ∇_{w_2} decreases faster: $16 \to 3.2 \to 0.64$ (80% reduction)
  - ∇_{w_1} decreases slower: $8 \to 6.4 \to 5.1$ (36% reduction)
- Problem with fixed LR:
  - Early iterations: w_2 takes huge steps (needs smaller LR)
  - Later iterations: w_1 still needs progress (needs maintained LR)
- Adaptive solution: Scale LR inversely with gradient history

Table of Contents

AdaGrad: Per-Parameter Learning Rates

- Key Idea: Adapt learning rate for each parameter based on historical gradients
  - Parameters with large gradients $\Rightarrow$ smaller effective learning rate
  - Parameters with small gradients $\Rightarrow$ larger effective learning rate
- Update Rules:

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t$$

  where $\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t)$ and $\odot$ is element-wise multiplication
- Hyperparameter: $\epsilon = 10^{-8}$ for numerical stability
- Problem: Learning rate decreases monotonically
  - $\mathbf{s}_t$ accumulates squared gradients without bound
  - Eventually $\eta/\sqrt{\mathbf{s}_t} \to 0$ and learning stops
- Use Case: Sparse features, NLP with large vocabularies

RMSprop: Fixing AdaGrad

- Problem with AdaGrad: Accumulated sum $\mathbf{s}_t$ grows forever
- RMSprop Solution: Use exponential moving average instead

$$\mathbf{s}_t = \gamma \mathbf{s}_{t-1} + (1-\gamma)\mathbf{g}_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}}\mathbf{g}_t$$

- Key Differences from AdaGrad:
  - $\mathbf{s}_t$ doesn't grow indefinitely (exponential decay)
  - Recent gradients weighted more heavily than old ones
  - Learning rate doesn't vanish over time
- Hyperparameters:
  - $\gamma = 0.9$ (decay rate for moving average)
  - $\eta = 0.001$ (learning rate)
  - $\epsilon = 10^{-8}$ (numerical stability)
- Use Case: RNNs, non-stationary problems, online learning

Algorithm: RMSprop

Algorithm 3: RMSprop

Data: Learning rate $\eta = 0.001$, decay rate $\gamma = 0.9$, $\epsilon = 10^{-8}$

Data: Initial parameters θ

Initialize accumulated gradient $\mathbf{s} = 0$

while *stopping criterion not met* **do**

  for *each mini-batch* $\mathcal{B}$ **do**

    Compute gradient: $\mathbf{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta)$

    Accumulate squared gradient: $\mathbf{s} \leftarrow \gamma \mathbf{s} + (1 - \gamma) \mathbf{g} \odot \mathbf{g}$

    Update parameters: $\theta \leftarrow \theta - \frac{\eta}{\sqrt{\mathbf{s} + \epsilon}} \odot \mathbf{g}$

- Good empirical performance across many problems
- Still missing: no momentum term

RMSprop: Numerical Example

Same problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\gamma = 0.9$, $\epsilon = 10^{-8}$

Iteration 0: $s_1 = s_2 = 0$

$$\text{Loss} = w_1^2 + 4w_2^2 = 4^2 + 4(2^2) = 16 + 16 = 32$$

$$g_1 = \nabla w_1 = 2w_1 = 2(4) = 8$$

$$g_2 = \nabla w_2 = 8w_2 = 8(2) = 16$$

$$s_1 \leftarrow \gamma s_1 + (1-\gamma) g_1^2 = 0.9(0) + 0.1(8^2) = 0 + 6.4 = 6.4$$

$$s_2 \leftarrow \gamma s_2 + (1-\gamma) g_2^2 = 0.9(0) + 0.1(16^2) = 0 + 25.6 = 25.6$$

$$\text{eff\_}\eta_1 = \frac{\eta}{\sqrt{s_1 + \epsilon}} = \frac{0.05}{\sqrt{6.4}} = \frac{0.05}{2.53} = 0.020$$

$$\text{eff\_}\eta_2 = \frac{\eta}{\sqrt{s_2 + \epsilon}} = \frac{0.05}{\sqrt{25.6}} = \frac{0.05}{5.06} = 0.010$$

$$w_1 \leftarrow w_1 - \text{eff\_}\eta_1 \cdot g_1 = 4 - 0.02(8) = 4 - 0.16 = 3.84$$

$$w_2 \leftarrow w_2 - \text{eff\_}\eta_2 \cdot g_2 = 2 - 0.01(16) = 2 - 0.16 = 1.84$$

RMSprop: Numerical Example

- Same problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\gamma = 0.9$, $\epsilon = 10^{-8}$

Iter	Loss	$\nabla_{w_1}, \nabla_{w_2}$	$s_1 \leftarrow$	$s_2 \leftarrow$	eff η_1	eff η_2	$w_1 \leftarrow$	$w_2 \leftarrow$
0	32.00	$[8, 16]$	6.4	25.6	0.020	0.010	3.84	1.84
1	28.30	$[7.68, 14.72]$	11.7	44.7	0.015	0.0075	3.72	1.73
2	25.80	$[7.44, 13.84]$	16.0	59.3	0.0125	0.0065	3.63	1.64
3	23.93	$[7.26, 13.12]$	19.7	70.6	0.011	0.0060	3.55	1.56

- Key observation:
  - Larger gradients (w_2) $\rightarrow$ larger s_2 $\rightarrow$ smaller effective LR (automatic adaptation!)
  - Loss decreases steadily: 32 $\rightarrow$ 28.30 $\rightarrow$ 25.80 $\rightarrow$ 23.93
- Issue: Slow convergence compared to momentum (no velocity accumulation)
- Iter 3 comparison: Mini-batch GD: 4.21, Momentum: 8.43, RMSprop: 23.93

RMSprop vs Momentum: What's Missing?

Feature	Momentum	RMSprop
Accumulates velocity?	✓	×
Smooths oscillations?	✓	×
Per-parameter learning rates?	×	✓
Adapts to gradient history?	×	✓

- Momentum: Great for smoothing, but fixed LR for all parameters
- RMSprop: Great for adaptation, but no velocity accumulation
- Question: Can we get the best of both?
- Answer: **Yes! Adam combines both ideas**
  - First moment m (like momentum): accumulates gradients
  - Second moment v (like RMSprop): accumulates squared gradients
  - Result: smooth updates with per-parameter adaptation

Adam: Adaptive Moment Estimation

- Key Idea: Combine momentum (first moment) + RMSprop (second moment)
- Update Rules :

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t \quad \text{(first moment: momentum)}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2 \quad \text{(second moment: adaptive LR)}$$

$$\theta_{t+1} = \theta_t - \frac{\eta \mathbf{m}_t}{\sqrt{\mathbf{v}_t} + \epsilon}$$

- Default Hyperparameters:
  - $\beta_1 = 0.9$ (momentum decay)
  - $\beta_2 = 0.999$ (RMSprop decay)
  - $\eta = 0.001$ or 0.0003 (learning rate)
  - $\epsilon = 10^{-8}$ (numerical stability)
- Note: Full Adam includes bias correction (shown in next slide)

Algorithm: Adam

Algorithm 4: Adam

Data: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Data: Initial parameters θ

Initialize first moment $\mathbf{m} = 0$, second moment $\mathbf{v} = 0$, timestep $t = 0$

while *stopping criterion not met* **do**

  for *each mini-batch* $\mathcal{B}$ **do**

    $t \leftarrow t + 1$

    Compute gradient: $\mathbf{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta)$

    Update first moment: $\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1)\mathbf{g}$

    Update second moment: $\mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2)\mathbf{g}^2$

    Bias correction: $\hat{\mathbf{m}} = \mathbf{m}/(1 - \beta_1^t)$, $\hat{\mathbf{v}} = \mathbf{v}/(1 - \beta_2^t)$

    Update parameters: $\theta \leftarrow \theta - \eta \hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}} + \epsilon)$

Adam: Numerical Example (Iteration 0)

Same problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
Iteration 0:

$$m_1 = m_2 = 0, \quad v_1 = v_2 = 0, \quad t = 1$$

$$\text{Loss} = w_1^2 + 4w_2^2 = 4^2 + 4(2^2) = 32$$

$$g_1 = \nabla_{w_1} = 2w_1 = 2(4) = 8$$

$$g_2 = \nabla_{w_2} = 8w_2 = 8(2) = 16$$

$$m_1 \leftarrow \beta_1 m_1 + (1 - \beta_1) g_1$$
$$= 0.9(0) + 0.1(8) = 0.8$$

$$m_2 \leftarrow \beta_1 m_2 + (1 - \beta_1) g_2$$
$$= 0.9(0) + 0.1(16) = 1.6$$

$$v_1 \leftarrow \beta_2 v_1 + (1 - \beta_2) g_1^2$$
$$= 0.999(0) + 0.001(64) = 0.064$$

$$v_2 \leftarrow \beta_2 v_2 + (1 - \beta_2) g_2^2$$
$$= 0.999(0) + 0.001(256) = 0.256$$

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^t} = \frac{0.8}{1 - 0.9^1} = \frac{0.8}{0.1} = 8$$

$$\hat{m}_2 = \frac{m_2}{1 - \beta_1^t} = \frac{1.6}{0.1} = 16$$

$$\hat{v}_1 = \frac{v_1}{1 - \beta_2^t} = \frac{0.064}{1 - 0.999^1} = \frac{0.064}{0.001} = 64$$

$$\hat{v}_2 = \frac{v_2}{1 - \beta_2^t} = \frac{0.256}{0.001} = 256$$

$$w_1 \leftarrow w_1 - \eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \epsilon} = 4 - 0.05 \times \frac{8}{\sqrt{64}} = 3.95$$

$$w_2 \leftarrow w_2 - \eta \frac{\hat{m}_2}{\sqrt{\hat{v}_2} + \epsilon} = 2 - 0.05 \times \frac{16}{\sqrt{256}} = 1.95$$

Adam: Numerical Example

- Problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$
- Hyperparameters: $\eta = 0.05$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Iter	Loss	∇	m_1, m_2	v_1, v_2	$\hat{m}_1, \hat{m}_2$	$\hat{v}_1, \hat{v}_2$	$w_1 \leftarrow$	$w_2 \leftarrow$
0	32.00	[8, 16]	[0.8, 1.6]	[0.064, 0.256]	[8, 16]	[64, 256]	$4 - 0.05 = 3.95$	$2 - 0.05 = 1.95$
1	30.81	[7.9, 15.6]	[1.51, 3.0]	[0.126, 0.499]	[7.95, 15.79]	[63.2, 249.6]	$3.95 - 0.05 = 3.90$	$1.95 - 0.05 = 1.90$
2	29.65	[7.8, 15.2]	[2.14, 4.22]	[0.187, 0.730]	[7.89, 15.57]	[62.4, 243.5]	$3.90 - 0.05 = 3.85$	$1.90 - 0.05 = 1.85$
3	28.51	[7.7, 14.8]	[2.70, 5.28]	[0.246, 0.948]	[7.84, 15.35]	[61.6, 237.3]	$3.85 - 0.05 = 3.80$	$1.85 - 0.05 = 1.80$

- Observation: Steady decrease: $32 \rightarrow 30.81 \rightarrow 29.65 \rightarrow 28.51$
- Notice: Updates are nearly constant (0.05 per iteration) due to bias correction balancing momentum and adaptive rates
- Iter 3 comparison ($\eta = 0.05$): Mini-batch: 4.21, Momentum: 8.43, RMSprop: 23.93, Adam: 28.51

Adam: Why So Popular?

- Bias Correction:
  - Ensures unbiased estimates, especially in early iterations
- What Adam Provides:
  - Momentum for faster convergence and smoothing
  - Individual adaptive learning rates per parameter
- Why It's the Default Choice:
  - Works well out-of-the-box with minimal tuning
  - Robust across different problem types
  - Combines multiple successful ideas
  - Good performance on wide range of architectures
- When to Prefer SGD+Momentum:
  - Production systems where you can afford careful tuning
  - Computer vision tasks (sometimes better final performance)
  - When you need best possible generalization

AdamW: Decoupled Weight Decay

- Difference from Adam: Separates weight decay from gradient-based update
- Standard Adam with L2 regularization:
  - L2 penalty added to loss: $\mathcal{L}_{total} = \mathcal{L} + \frac{\lambda}{2}\|\theta\|^2$
  - Gradient includes regularization term
  - Interacts with adaptive learning rates in unintended ways
- AdamW Update:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda \theta_t \right)$$

  - Weight decay applied directly to parameters
  - Decoupled from gradient-based adaptive update
- Hyperparameters:
  - Same as Adam: $\beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001, \lambda = 0.01$ (weight decay)
- When to use: When combining optimization with regularization, often better than Adam with L2

Adam vs AdamW: Side-by-Side Comparison

Aspect	Adam	AdamW
L2 Regularization	Added to loss: $\mathcal{L} + \frac{\lambda}{2}\|\theta\|^2$	Applied directly to weights
Gradient	Includes reg. term: $\nabla\mathcal{L} + \lambda\theta$	Only from loss: $\nabla\mathcal{L}$
Update Rule	$\theta \leftarrow \theta - \frac{\eta\hat{m}}{\sqrt{\hat{v}}+\epsilon}$	$\theta \leftarrow \theta - \eta\left(\frac{\hat{m}}{\sqrt{\hat{v}}+\epsilon} + \lambda\theta\right)$
Adaptive Rates	Applied to gradient+reg	Applied only to gradient
Weight Decay	Depends on adaptive rates	Constant, decoupled
When to Use	General optimization	With L2 regularization
Performance	Good	Often better generalization

- Key insight: AdamW separates optimization from regularization
- Practical advice: Use AdamW when you need weight decay/L2 regularization

Table of Contents

Why Schedule Learning Rate?

- Problem with Fixed Learning Rate:
  - Early training: large gradients, need to explore broadly
  - Late training: near optimum, need fine-grained adjustments
  - Fixed LR: either too aggressive later or too conservative early
- Solution: Decrease learning rate over time
  - Start with larger LR for faster initial progress
  - Gradually reduce LR for stable convergence
- Common Strategies:
  - Step decay: drop by fixed factor at intervals
  - Exponential decay: smooth continuous reduction
  - Cosine annealing: smooth with possible restarts
  - Warmup: gradually increase then decay

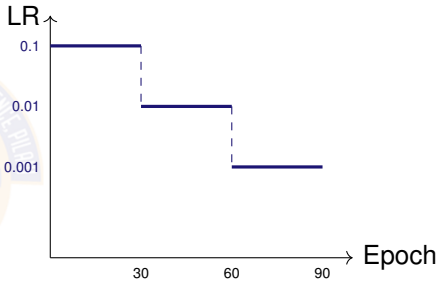
Step Decay Schedule

- Formula:

$$\eta_t = \eta_0 \times \gamma^{\lfloor t/\text{step\_size} \rfloor}$$

where t is the current epoch number

- Hyperparameters:
  - η_0: initial learning rate
  - γ: drop factor (typical: 0.1)
  - step_size: epochs between drops (typical: 30)
- When to use: Computer vision, simple and interpretable
- Example: $\eta_0 = 0.1$, $\gamma = 0.1$, step = 30
  - Epoch 10: $\eta = 0.1 \times 0.1^{\lfloor 10/30 \rfloor} = 0.1 \times 0.1^0 = 0.1$
  - Epoch 40: $\eta = 0.1 \times 0.1^{\lfloor 40/30 \rfloor} = 0.1 \times 0.1^1 = 0.01$



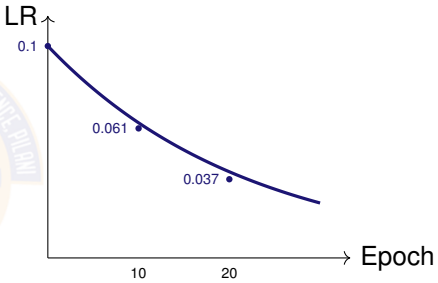
Exponential Decay Schedule

- Formula:

$$\eta_t = \eta_0 \times e^{-\lambda t}$$

where t is the current epoch number

- Hyperparameters:
  - η_0: initial learning rate
  - λ: decay rate (typical: 0.01-0.1)
- Advantages:
  - Smooth, continuous decay
  - No sudden drops
  - Easy to tune with single λ



- Example: $\eta_0 = 0.1$, $\lambda = 0.05$
  - Epoch 10: $\eta = 0.1 \times e^{-0.05(10)} = 0.1 \times e^{-0.5} = 0.1 \times 0.606 = 0.061$
  - Epoch 20: $\eta = 0.1 \times e^{-0.05(20)} = 0.1 \times e^{-1.0} = 0.1 \times 0.368 = 0.037$

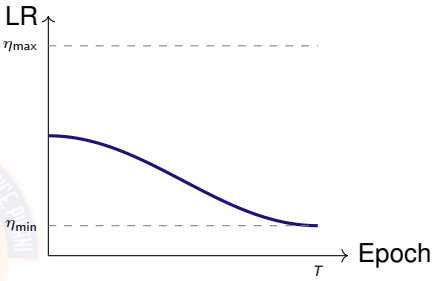
Cosine Annealing Schedule

- Formula:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{\pi t}{T}\right)\right)$$

where t is current epoch, T is total epochs

- Hyperparameters:
  - $\eta_{\max}$: initial/max LR
  - $\eta_{\min}$: minimum LR
  - T: total training epochs
- Advantages:
  - Very smooth decay
  - No decay rate tuning needed
  - Popular in modern deep learning



- Example: $\eta_{\max} = 0.1$, $\eta_{\min} = 0.001$, $T = 100$
  - Epoch 0: $\eta = 0.001 + \frac{0.099}{2}(1 + \cos(0)) = 0.1$
  - Epoch 50: $\eta = 0.001 + 0.0495(1 + \cos(\pi)) = 0.001$

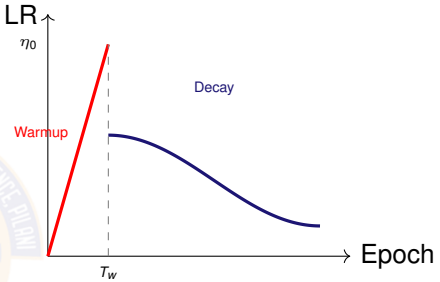
Warmup Strategy

- Concept: Gradually increase LR from 0 to target over initial epochs
- Linear Warmup:

$$\eta_t = \eta_0 \times \frac{t}{T_{\text{warmup}}}$$

for $t \leq T_{\text{warmup}}$

- Why Needed?
  - Large initial LR causes instability
  - Important for large batches/models
- Typical: 5-10 epochs warmup, then apply decay schedule



- Example: $\eta_0 = 0.1$, $T_{\text{warmup}} = 5$, then cosine to epoch 100
  - Epoch 2: $\eta = 0.1 \times \frac{2}{5} = 0.04$ (warmup)
  - Epoch 5: $\eta = 0.1 \times \frac{5}{5} = 0.1$ (warmup complete)
  - Epoch 6-100: Apply cosine annealing from 0.1 to 0.001

Table of Contents

Gradient Clipping

- Problem: Exploding gradients cause training instability
  - Gradients become very large (NaN, Inf values)
  - Weight updates are too large
  - Loss diverges or oscillates wildly
- Solution: Gradient Clipping
  - Norm-based Clipping
  - Value-based Clipping
- When to use: RNNs, transformers, unstable training, very deep networks

Gradient Clipping

- Norm-based Clipping:

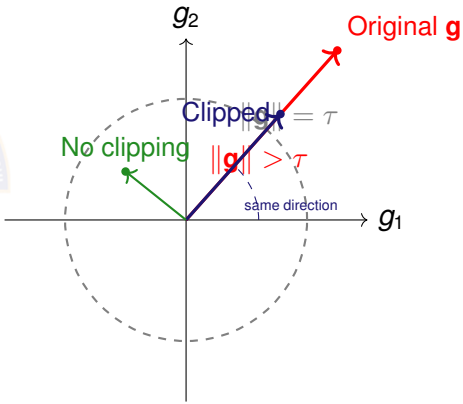
$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \leq \tau \\ \tau \frac{\mathbf{g}}{\|\mathbf{g}\|} & \text{if } \|\mathbf{g}\| > \tau \end{cases}$$

  - Hyperparameter: τ (typical: 1.0-5.0)
  - Preserves gradient direction
  - Most commonly used

- Value-based Clipping:

$$g_i \leftarrow \text{clip}(g_i, -\tau, \tau)$$

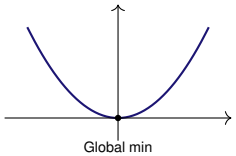
  - Clips each gradient component
  - Hyperparameter: τ (typical: 0.5-1.0)
  - May change gradient direction



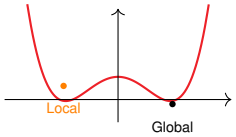
Loss Landscape Visualization

- Convex Loss:
  - Single bowl shape
  - Any local minimum = global minimum
  - Example: Linear regression
  - Easy to optimize
- Non-Convex Loss:
  - Multiple valleys, peaks, saddles
  - Many local minima
  - Example: Deep neural networks
  - Requires sophisticated optimizers
- Why Visualize?
  - Understand optimizer behavior
  - Identify problematic regions
  - Guide hyperparameter choices

Convex:



Non-Convex:

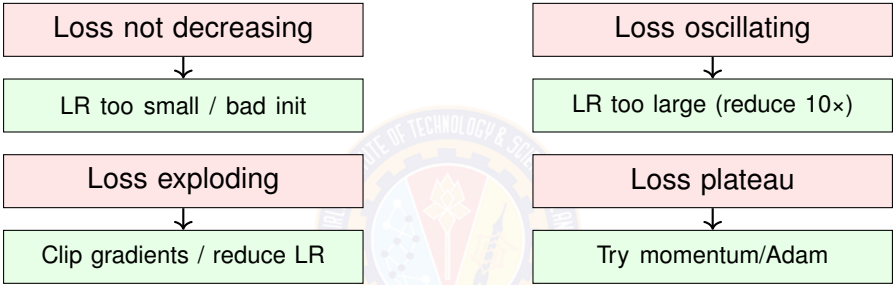


Batch Size Effects on Optimization

Aspect	Small Batch (16-32)	Large Batch (256-1024)
Gradient noise	High	Low
Updates/epoch	Many (N/B)	Few (N/B)
Convergence	Noisy but explores well	Smooth but may get stuck
Generalization	Often better	May need LR scaling
Training time	Slower (more updates)	Faster (GPU efficient)
Memory usage	Low	High
Learning rate	Can use larger	May need warmup

- Linear Scaling Rule: When doubling batch size, double learning rate
  - Maintains similar convergence speed
- Practical Recommendation:
  - Start with batch size 64-128
  - Increase if GPU memory allows and training is slow
  - Use warmup when increasing batch size

Debugging Training Issues



- General Debugging Checklist:
  - Plot training and validation loss curves
  - Monitor gradient norms: check for NaN/Inf
  - Verify data loading and preprocessing
  - Try overfitting small batch (sanity check)

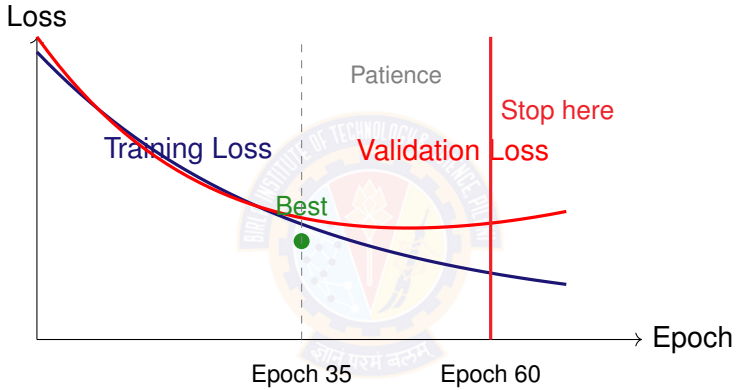
Stopping Criteria

- Common Stopping Criteria:

Criterion	Description	Typical Value
Max epochs	Predetermined training limit	100-200 epochs
Early stopping	Validation loss stops improving	Patience: 5-10 epochs
Gradient threshold	Gradient norm below threshold	$\|\nabla \mathcal{L}\| < 10^{-6}$
Loss threshold	Loss reaches target value	Problem-specific

- Early Stopping (Most Common):
  - Track best validation loss
  - If no improvement for p epochs (patience), stop
  - Restore model to best validation checkpoint
  - Prevents overfitting
- Practical Strategy: Combine max epochs + early stopping
  - Set max epochs as upper bound
  - Use early stopping to terminate earlier if converged

Early Stopping: Visualization



- Best model: Epoch 35 (lowest validation loss)
- Patience: Wait 25 more epochs with no improvement
- Stop: At epoch 60, restore model from epoch 35

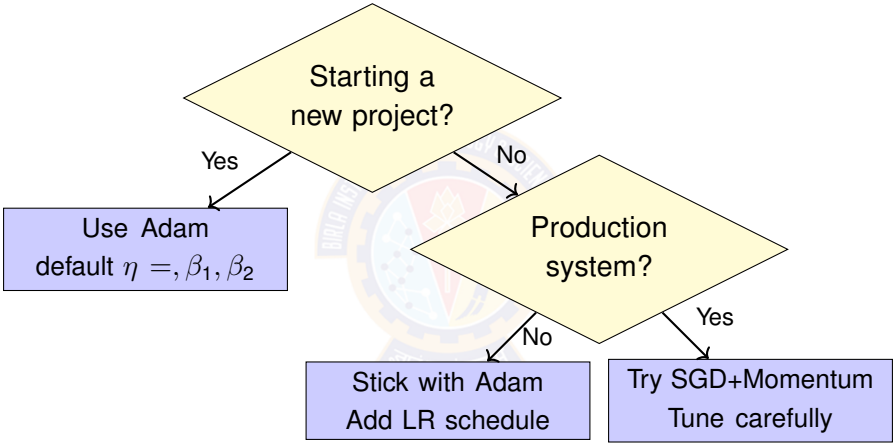
Table of Contents

Optimizer Comparison Summary

Method	Convergence	Memory	Tuning	Stability
SGD	Slow	Low	Hard	High
SGD+Momentum	Medium	Low	Medium	High
AdaGrad	Good	Medium	Easy	Medium
RMSprop	Good	Medium	Easy	Medium
Adam	Fast	Medium	Very Easy	High
AdamW	Fast	Medium	Very Easy	High

Optimizer	Best Use Cases
Adam/AdamW	Default choice, quick experiments, NLP, most problems
SGD+Momentum	Production CV, when you can afford tuning, best final performance
RMSprop	RNNs, online learning, non-stationary problems
AdaGrad	Sparse features, NLP with large vocabularies

When to Use Which Optimizer



- For best results: Try SGD+Momentum with careful tuning
- Special cases: RMSprop for RNNs, AdaGrad for sparse data

Hyperparameter Summary Table

Optimizer	Key Hyperparameters	Typical Values	Sensitivity
SGD	η	0.01-0.1	High
SGD+Momentum	η, β	0.01-0.1, 0.9	Medium
AdaGrad	η, ϵ	0.01, 10^{-8}	Medium
RMSprop	η, γ, ϵ	0.001, 0.9, 10^{-8}	Medium
Adam	$\eta, \beta_1, \beta_2, \epsilon$	0.001, 0.9, 0.999, 10^{-8}	Low
AdamW	$\eta, \beta_1, \beta_2, \epsilon, \lambda$	0.001, 0.9, 0.999, 10^{-8}, 0.01	Low

Other Hyperparameters	Typical Values
Batch size	32, 64, 128, 256 (powers of 2)
Warmup epochs	5-10 (or 5-10% of total)
Gradient clipping threshold	1.0-5.0 (norm) or 0.5-1.0 (value)
Early stopping patience	5-10 epochs

How to Tune Hyperparameters

- Priority Order:

1. Learning Rate	Grid search: [1e-5, 1e-4, 1e-3, 1e-2]
2. Batch Size	Try: 32 $\rightarrow$ 64 $\rightarrow$ 128
3. LR Schedule	Add warmup + cosine
4. Optimizer-specific	Usually keep defaults

- Learning Rate Tuning:
  - Start with optimizer default (0.001 for Adam, 0.01 for SGD)
  - If loss explodes: reduce by 10×
  - If too slow: increase by 3-5×
  - Use LR range test for systematic search

Hyperparameter Interactions

- Learning Rate × Batch Size:
  - Linear scaling rule: Double batch $\rightarrow$ Double LR
  - Example: Batch 32 ($\eta = 0.01$) $\rightarrow$ Batch 256 ($\eta = 0.08$)
  - Large batches need warmup to stabilize
- Learning Rate × Optimizer:
  - Adam: Less sensitive, $\eta = 0.001$ works broadly
  - SGD: Very sensitive, needs careful tuning
  - Momentum amplifies steps $\rightarrow$ needs smaller LR than vanilla SGD
- Batch Size × Generalization:
  - Small batches (32-64): Noisy gradients $\rightarrow$ regularization effect $\rightarrow$ better generalization
  - Large batches (256+): Smooth gradients $\rightarrow$ may overfit $\rightarrow$ needs LR warmup
- LR Schedule × Optimizer:
  - Adam + aggressive schedule: Can hurt performance
  - SGD benefits more from schedules than Adam

Implementation Best Practices

- Workflow:
  1. Start with Adam ($\eta = 0.001$), default hyperparameters
  2. Monitor training curves (train loss, val loss, gradient norms)
  3. Add cosine annealing schedule
  4. Apply gradient clipping if training unstable
  5. Use warmup for large models/batches
  6. Implement early stopping (patience=10)
  7. Save checkpoints regularly
- Sanity Checks:
  - Overfit small batch first (10-100 examples)
  - Verify loss decreases on single batch
  - Check gradient norms are reasonable (not NaN/Inf)
  - Visualize first few batches of data
- Common Pitfalls:
  - Not monitoring training curves
  - Using wrong default LR (0.01 for Adam instead of 0.001)
  - Batch size too large without LR scaling
  - Training too long without early stopping

Table of Contents

Key Takeaways

1. Adam is the default choice
   - Works well out-of-the-box: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$
   - Robust to hyperparameter choices
   - Combines momentum + adaptive learning rates
2. SGD+Momentum for production
   - Needs careful tuning but often better final performance
   - Popular in computer vision
   - Use when you have time and resources to tune
3. Learning rate is most important
   - Tune this first before anything else
   - Too large: divergence; Too small: slow training
   - Use schedules (cosine annealing) for better convergence
4. Batch size matters
   - Start with 64-128
   - Small batches: better generalization
   - Large batches: need LR scaling + warmup

Key Takeaways (Continued)

5. Use LR schedules + warmup
   - Cosine annealing: smooth, no tuning needed
   - Warmup (5-10 epochs): prevents early instability
   - Especially important for large models/batches
6. Gradient clipping for stability
   - Essential for RNNs, transformers
   - Norm clipping (threshold: 1-5) most common
   - Prevents exploding gradients
7. Monitor everything
   - Plot train/validation loss curves
   - Track gradient norms
   - Use early stopping (patience: 10 epochs)
   - Save checkpoints regularly
8. No universal solution
   - Different problems need different approaches
   - Understand trade-offs and experiment
   - Start simple (Adam), add complexity if needed

References

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning*. Cambridge University Press. Chapters 12.
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.

Thank You!